

MOSCOW AVIATION INSTITUTE
(NATIONAL RESEARCH UNIVERSITY)

Institute No. 8 “Computer Science and Applied Mathematics”
Department 806 “Computational Mathematics and Programming”

Laboratory work No. 1
Course “Computer Graphics”

3D Graphics Basics

Student: M. A. Khorokhorin
Group: M8O-306BV-24
Teacher: V. D. Bakharev

Moscow, 2026

Task

Goal: learn the basics of 3D graphics — building simple 3D objects, projecting them onto a 2D plane, perspective and orthographic projection matrices and affine transformations. The program must run in real time on Vulkan, GLFW and ImGui and let the user change the projection and the transformations of the objects.

Variant 4: a truncated regular tetrahedron, whose faces are 4 regular triangles and 4 regular hexagons.

All extra tasks are done: switching between orthographic and perspective projection; changing the position, rotation and scale of the object; movement and rotation along a trajectory with pause, play, speed and trajectory parameters; an object color picker; procedural vertex colors multiplied by the picked color; several objects, each with its own descriptor set.

Method

Geometry. The regular tetrahedron has the corners $(1, 1, 1)$, $(1, -1, -1)$, $(-1, 1, -1)$, $(-1, -1, 1)$. Every edge A_iA_j is split into three parts, the point near A_i is $P_{ij} = (2A_i + A_j) / 3$. This gives 12 vertices, 18 edges and 8 faces ($V - E + F = 2$). Every corner of the tetrahedron is cut off as a triangle and every original face becomes a hexagon. The vertices are normalized, so the object fits into the unit sphere. Hexagons are split into 4 triangles as a fan, 20 triangles and 60 indices in total. The winding of each face is chosen so that the normal $(b - a) \times (c - a)$ points outwards.

The vertex color comes from the local position: $\text{color} = \text{position} \cdot 0.5 + 0.5$. The vertex shader multiplies it by the color picked in the interface; with vertex colors turned off only the picked color remains. To keep faces of the same color apart, the fragment shader applies flat shading: the face normal is the cross product of the screen-space derivatives of the view-space position ($dFdx \times dFdy$), and the brightness follows Lambert's law: $0.35 + 0.65 \cdot \max(n \cdot l, 0)$.

Transformations. The model matrix is $M = T \cdot R_y \cdot R_x \cdot R_z \cdot S$: scale first, then rotations, then translation. The camera orbits the origin and the view matrix is built with `lookAt`. The projection matrices are written by hand for the Vulkan coordinate system, where the Y axis points down and depth is in $[0, 1]$. For the perspective projection with $f = 1 / \tan(\text{fov} / 2)$: $x' = f \cdot x / a$, $y' = -f \cdot y$, $z' = \text{far} \cdot z / (\text{near} - \text{far}) + \text{near} \cdot \text{far} / (\text{near} - \text{far})$, $w' = -z$, so the near plane maps to depth 0 and the far plane to 1. Orthographic projection: $x' = x / (h \cdot a)$, $y' = -y / h$, $z' = (z + \text{near}) / (\text{near} - \text{far})$, $w' = 1$, the size does not depend on the distance.

Animation. The object moves along an epitrochoid with a vertical wave: $x = R \cdot \cos t + r \cdot \cos kt$, $y = H \cdot \sin 2t$, $z = R \cdot \sin t + r \cdot \sin kt$, and spins with the angular speed spin . The animation time accumulates as $t += dt \cdot \text{speed}$ only while playing, so pausing and changing the speed do not cause jumps.

Vulkan objects. The vertex and index buffers (`VkBuffer`) are allocated with Vulkan Memory Allocator in host-visible memory and filled with `memcpy`. Uniform buffers hold the camera (view, projection) and every object (model, color, vertex color and shading weights). The C++ structures follow the `std140` layout, which is checked with `static_assert`. The program creates two `VkDescriptorSetLayout` objects, a `VkDescriptorPool` for 9 sets, set 0 for the camera and eight sets 1 for the objects, a `VkPipelineLayout` with two sets and a `VkPipeline`: one vertex buffer with position and color, a triangle list, back-face culling, depth test and write, dynamic viewport and scissor. GLSL shaders are compiled to SPIR-V with `glslc`. The main `VkRenderPass` of the starter code got an external subpass dependency (`VkSubpassDependency`) that orders the layout transition of the swapchain image after it is acquired; without it synchronization validation reported a `WRITE_AFTER_READ` hazard.

Frame. `update` draws the ImGui interface and computes the matrices on the CPU. `render`, after the fence of the previous frame is waited on, copies the data to the uniform buffers and records the command buffer: begin the render pass clearing color and depth, set the viewport and scissor, bind the pipeline, the buffers and the camera set, then for every object bind its set and call `vkCmdDrawIndexed`.

Geometry and matrices live in a separate library without Vulkan dependencies and are covered with GoogleTest unit tests. The program was checked with the Vulkan validation layers, including synchronization and best practices validation: there are no messages.

Results

The figures show the program: the scene settings window on the left (projection, camera, object list), the selected object window on the right (transform, color, animation) and the scene in the middle.

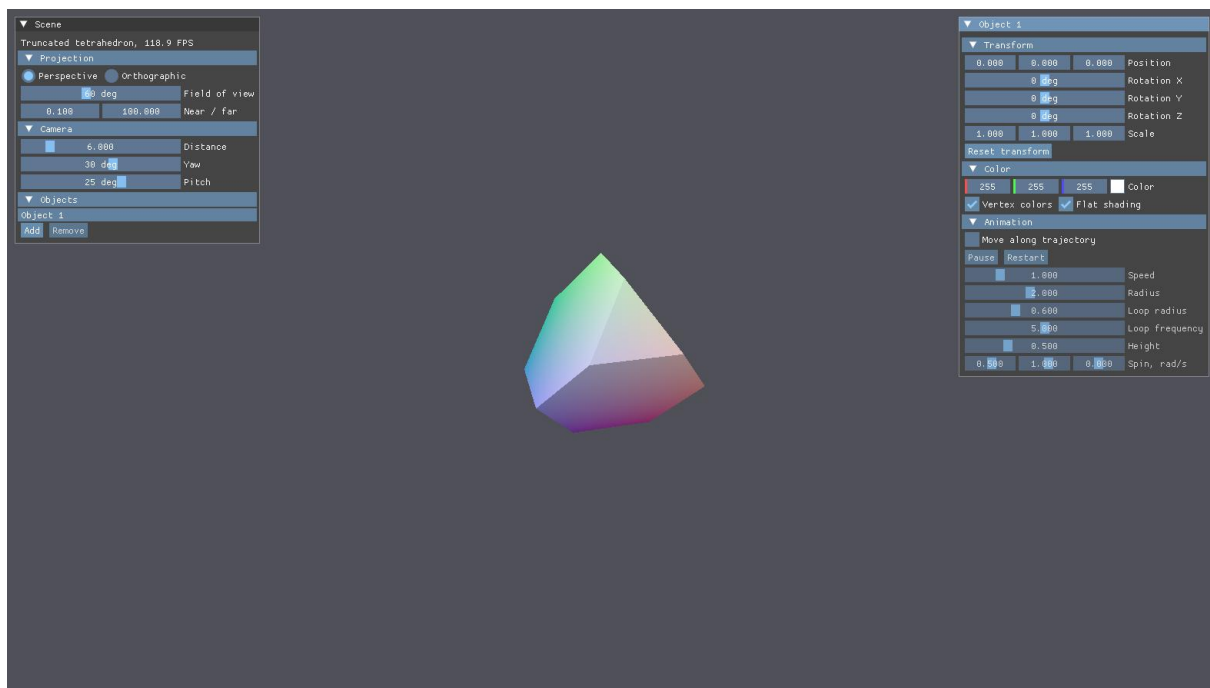


Figure 1 — perspective projection with vertex colors

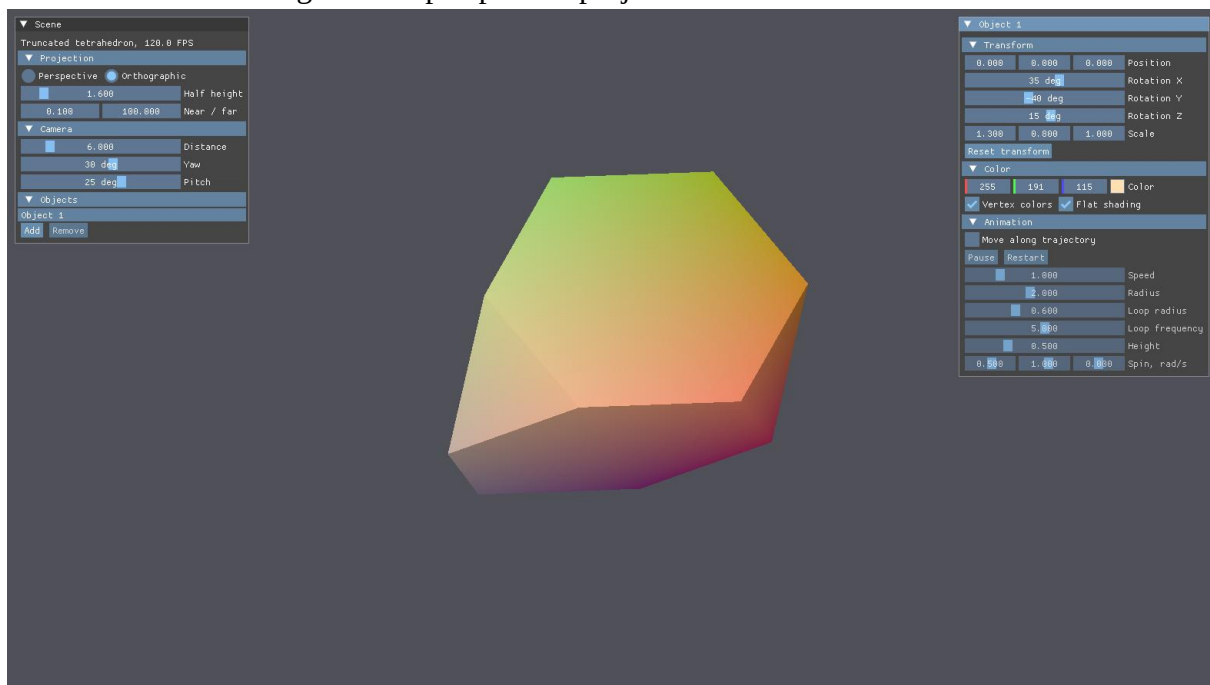


Figure 2 — orthographic projection with changed rotation, scale and color

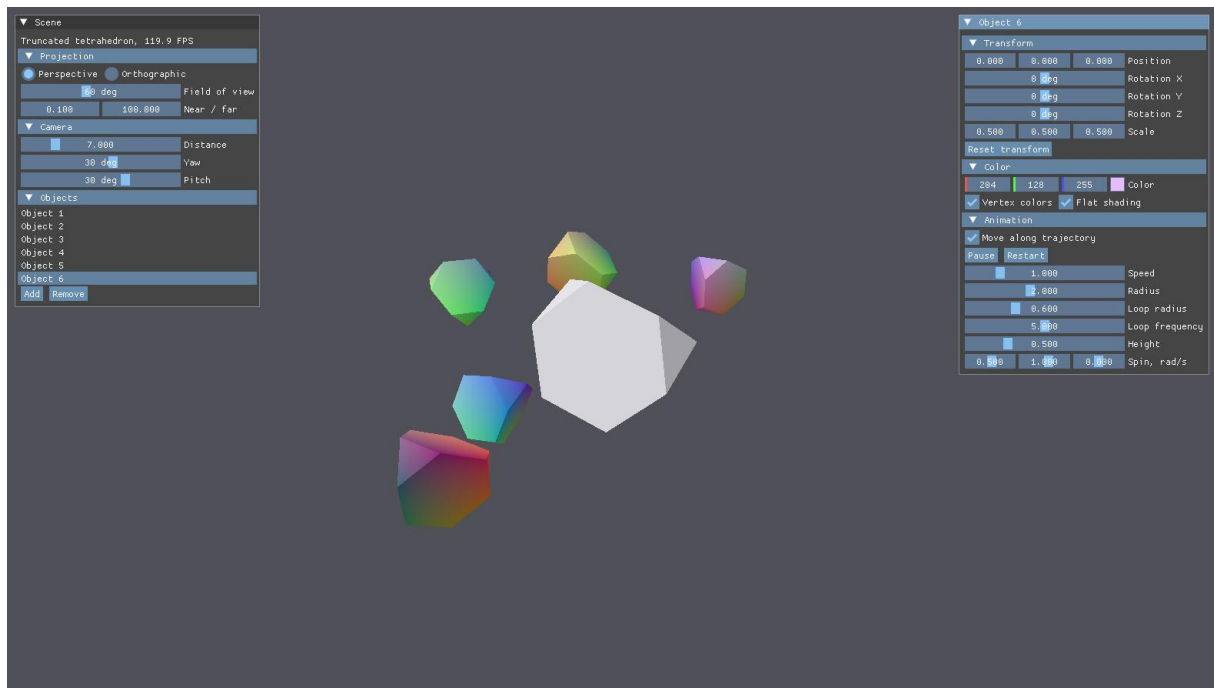


Figure 3 — several objects moving along the trajectory

Conclusions

I learned to build a 3D object from vertices and indices, pass data to shaders through vertex and uniform buffers, describe the Vulkan graphics pipeline and descriptor sets, derive perspective and orthographic projection matrices for the Vulkan coordinate system and compose affine transformations. I also understood why the triangle winding matters for back-face culling and how to align data according to std140.